

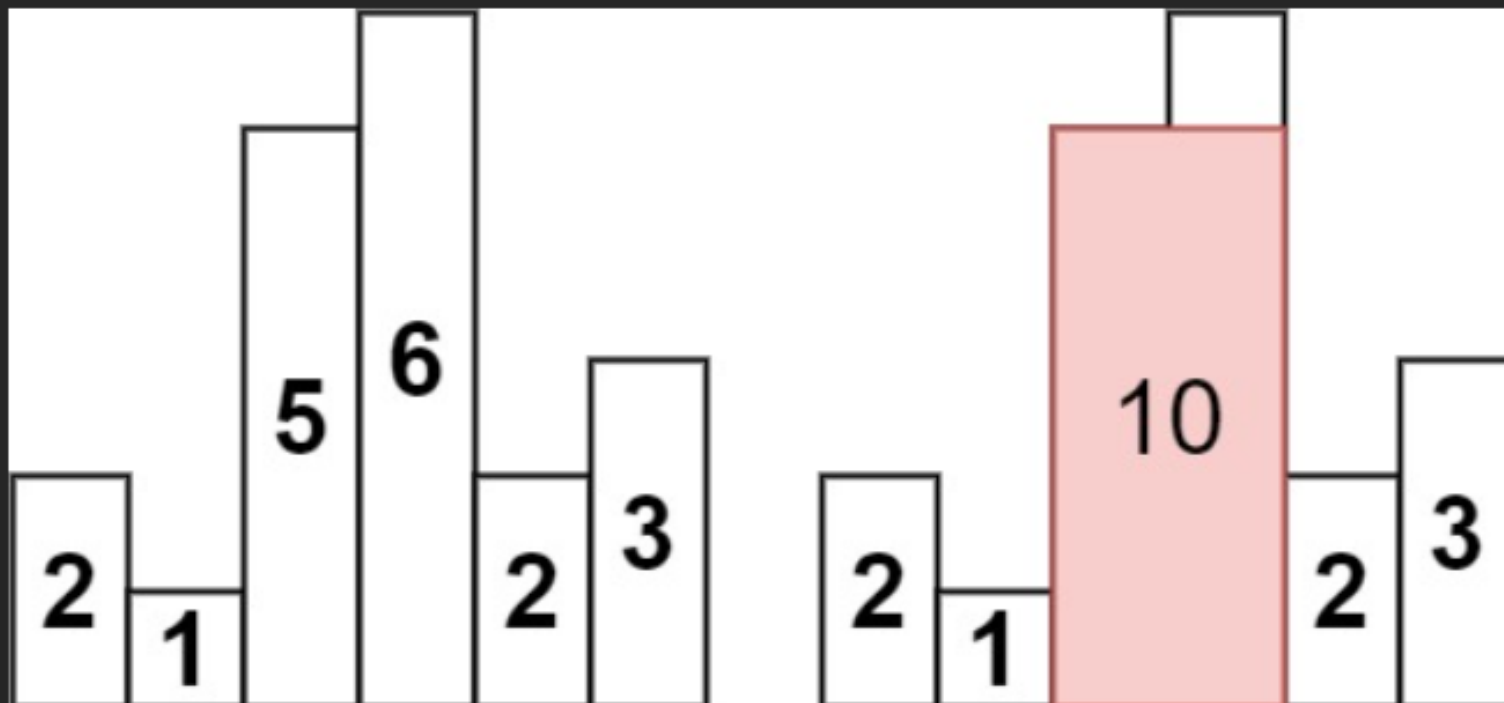
84. Largest Rectangle in Histogram

Solved 

Hard

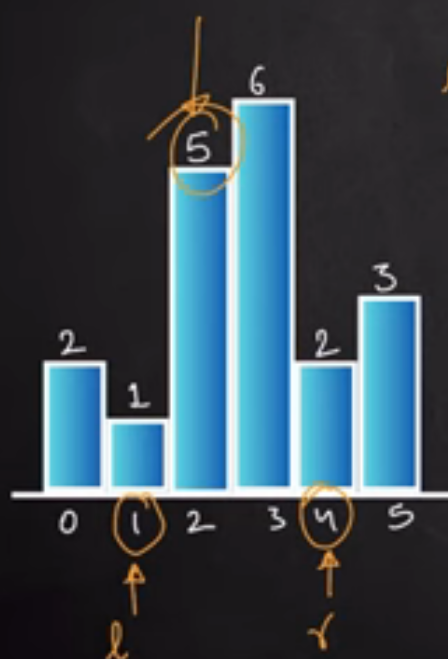
 Topics Companies

Given an array of integers `heights` representing the histogram's bar height where the width of each bar is `1`, return *the area of the largest rectangle in the histogram*.

Example 1:**Input:** `heights = [2,1,5,6,2,3]`**Output:** 10

Explanation: The above is a histogram where width of each bar is 1. The largest rectangle is shown in the red area, which has an area = 10 units.

Largest Rectangle in Histogram

`heights = [2, 1, 5, 6, 2, 3]`

left smaller nearest $\leftarrow$ $\rightarrow$ right smaller nearest

 $width = r - l - 1$ $ans = 0$

```
for (i=0 to n) {
    curArea = height[i] * (r-l-1)
    ans = max(ans, curArea)
}
```

```
return ans
```



same concept as next greater on right = right greater nearest

we need right smaller nearest and left smaller nearest

```

int largestRectangleArea(vector<int>& heights) {
    int n = heights.size();

    vector<int> right(n) , left(n);

    stack<int> st;

    for(int i=n-1;i>=0;i--){
        int ele = heights[i];
        while(!st.empty() && ele <= heights[st.top()]){
            st.pop();
        }

        if(st.empty()){
            right[i] = n;
        }
        else{
            right[i] = st.top();
        }

        st.push(i);
    }

    while(!st.empty()) st.pop();

    while(!st.empty()) st.pop();

    for(int i=0;i<n;i++){
        int ele = heights[i];
        while(!st.empty() && ele <= heights[st.top()]){
            st.pop();
        }

        if(st.empty()){
            left[i] = -1;
        }
        else{
            left[i] = st.top();
        }

        st.push(i);
    }

    int ans = 0;
    for(int i=0;i<n;i++){
        ans = max(ans , heights[i]*(right[i]-left[i]-1));
    }

    return ans;
}

```

**

85. Maximal Rectangle

Solved ✓

Hard

Topics

Companies

Given a `rows x cols` binary `matrix` filled with `0`'s and `1`'s, find the largest rectangle containing only `1`'s and return *its area*.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: `matrix = [ ["1","0","1","0","0"], ["1","0","1","1","1"], ["1","1","1","1","1"], ["1","0","0","1","0"] ]`

Output: 6

Explanation: The maximal rectangle is shown in the above picture.

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

- We can accumulate height at each row.
- Problem becomes (84)
- We call the function for heights at each row to get the max. Rectangle.

